Contents

CLI Launcher + Notes — High-Level Design **1**
1. Goals . . . 1
2. Success criteria . . . 2
3. System architecture . . . 2
4. Android application . . . 3
5. Desktop client (WPF) . . . 6
6. Azure backend . . . 7
7. Sync . . . 10
8. Evaluation plan . . . 11
9. Phased delivery . . . 12
10. Risks . . . 12
11. Open items . . . 13
Sources . . . 14

CLI Launcher + Notes — High-Level Design

Status: Draft v3 **Author:** Jack **Date:** 2026-08-13 **Scope:** Personal-use side project. Android CLI launcher with on-device SLM, native WPF desktop client, Azure free-tier backend.

Changes in v3: Note editor changed from a vim-style modal editor to a Google Keep-style touch editor rendered inline in the CLI surface (§4.6). Net reduction in scope.

Changes in v2: Desktop client moved from an Azure Static Web Apps SPA to a **native WPF (.NET) application**, and the backend is pinned to **C# on the Functions isolated worker model**. Rationale: a coherent end-to-end .NET story for a .NET job hunt, plus genuine offline capability on the desktop. Knock-on effects are non-trivial — see §7 (sync), §6.4 (auth), and §10 (risks).

---

1. Goals

Product goals

- Replace the Android home screen with a keyboard-first, CLI-style launcher.
- Launch apps, open settings, and create/edit notes and checklists without leaving the prompt.
- Read and edit the same notes from a native desktop client, **fully functional offline**.
- Free-form natural-language commands interpreted by a small language model running **entirely on the phone**.

Portfolio goals (these are first-class requirements, not side effects)

- Demonstrate a complete on-device GenAI pipeline: model selection → fine-tune → quantize → deploy → measure.
- Demonstrate **end-to-end .NET**: WPF + MVVM on the client, EF Core locally, C# isolated-worker Functions on the server, with domain and sync logic shared as a common library.
- Produce reproducible benchmarks (accuracy, latency, memory, battery).
- Demonstrate a cloud backend that is correct, secured, and operated inside a fixed $0 budget.

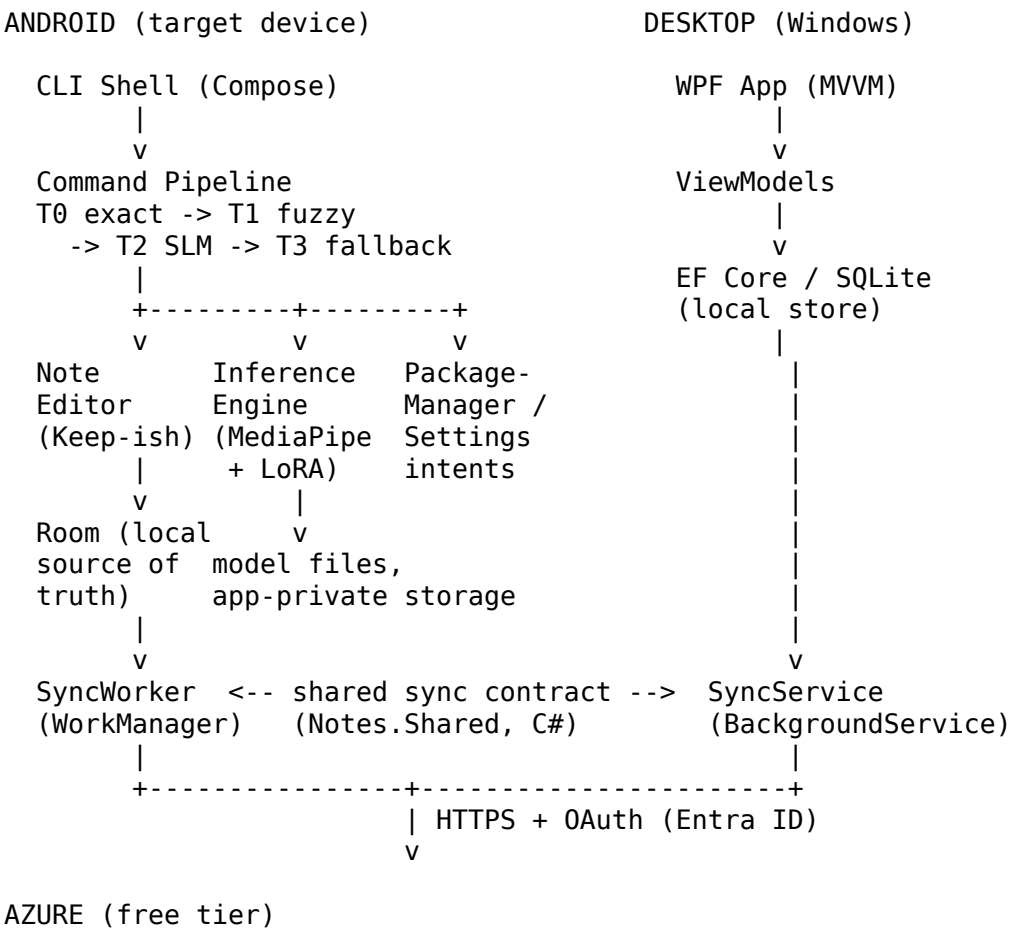
Non-goals

- Multi-user support, sharing, or collaboration.
- Broad device compatibility. One target Android device and Windows on the desktop is sufficient.
- Cross-platform desktop. WPF is Windows-only and that is accepted.
- Cloud LLM inference of any kind. If it can't run on the phone, it's out of scope.
- Real-time sync. Periodic and on-demand sync is sufficient.

2. Success criteria

Dimension	Target
Intent accuracy (held-out set)	≥ 95% on in-distribution commands
Schema-valid JSON rate	≥ 99% (after repair pass)
Tier 0/1 command latency	< 50 ms input-to-action
SLM end-to-end latency	< 1.5 s p50 on target device
Model resident memory	< 1.5 GB during inference
Cold app-open path	Never blocked on the SLM
Desktop offline capability	Full read/write with the network disabled; syncs cleanly on reconnect
Desktop cold start	< 2 s to an interactive, populated note list
Sync convergence	Both clients agree within one sync cycle after an offline edit on each
Azure spend	$0.00/month, verified by budget alert

3. System architecture



```
                        v
  Cosmos DB (NoSQL, free tier, provisioned throughput)
                        |
                        v
  Azure AI Language (F0) [optional]

  Blob Storage (free tier) --> SLM model + LoRA artifacts
```

Three deployables: Android app, WPF desktop app, Azure backend. Android carries the ML risk; the desktop now carries real sync risk, because there are **two independently offline-capable clients** rather than one (§7).

---

4. Android application

4.1 Launcher shell

Registered as a HOME activity (`ACTION_MAIN` + `CATEGORY_HOME` + `CATEGORY_DEFAULT`). Single full-screen Compose surface: scrollback transcript above, input line at the bottom, IME open by default.

- Dark-only theme. No light palette is defined, no theme switch exists.
- BACK is a no-op at the prompt (standard launcher behaviour); HOME re-focuses the input. **When the note editor overlay is open, `BACK` closes the overlay** rather than being swallowed — it is the only state where BACK does anything (§4.6).
- Scrollback is ephemeral and capped (last N entries, cleared on process death). It is not persisted or synced.

4.2 Command pipeline

The single most important design decision: **the SLM is one tier of a cascade, not the front door.** A launcher that waits on a language model to open a phone app is a worse launcher.

Tier	Handler	Latency	Handles
T0	Exact match — user aliases, reserved commands (`:apps`, `:settings`, `:note`, `:alias`)	< 5 ms	Muscle-memory input
T1	Fuzzy app-name match over the `PackageManager` index (normalized edit distance + prefix/acronym boost)	< 50 ms	`chr` → Chrome
T2	SLM intent + slot extraction	~1 s	Free-form natural language
T3	Fallback — treat as app-name search, else show help	—	Everything else

Escalation rule: T0 → T1 → T2 only when the prior tier's confidence is below threshold. T1 short-circuits on a single high-confidence match. If input contains whitespace and no tier-0/1 hit, go straight to T2 (multi-word input is almost always prose).

Two consequences worth designing for deliberately:

1. **T0/T1 hits are free training data.** Log (`raw_input, resolved_intent`) locally for every deterministic resolution. This bootstraps the fine-tuning set from real usage instead of pure synthesis.
2. **T3 is the safety net that makes T2 shippable.** The SLM is allowed to fail; the launcher is not.

4.3 Intent schema

Closed-world intent classification with slot filling — not open-ended generation. Fixed label space:

```
open_app, list_apps, open_settings, create_note, create_checklist,
open_note, search_notes, check_item, define_alias, help, unknown
```

Model output contract:

```
{
  "intent": "create_note",
  "slots": { "title": "call the bank", "body": "", "tags": ["personal"] },
  "confidence": 0.91
}
```

unknown is a first-class, *desirable* output. Training the model to abstain is cheaper and safer than training it to guess, and it routes cleanly to T3.

4.4 On-device inference

Runtime: MediaPipe LLM Inference API (`LlmInference` / `LlmInferenceOptions`), GPU backend.

Key config: `modelPath`, `maxTokens` (512 default is ample — commands are short), `temperature` set low (~0.1; this is extraction, not creative writing), `topK`, `loraPath`.

> ⚠ **Known risk, accepted deliberately.** Google has placed the MediaPipe LLM Inference API in **maintenance-only mode** and is directing new work to **LiteRT-LM**. This design uses MediaPipe as specified, but *all* inference sits behind an `InferenceEngine` interface (`suspend fun infer(prompt: String): String`, plus lifecycle) with `MediaPipeEngine` as the first implementation. A `LiteRtLmEngine` can be swapped in without touching the command pipeline. Doing that migration and benchmarking both backends is a strong portfolio artifact in its own right — budget it as an explicit v2 milestone rather than an emergency.

LoRA constraints that shape model choice: MediaPipe LoRA inference requires `backend = 'gpu'`, applies to attention layers only, and is documented as experimental. Compatible models are Gemma variants and Phi-2 (the Android guide names Gemma-2 2B, Gemma 2B, and Phi-2 explicitly). Adapters are trained with PEFT (`target_modules = ["q_proj","v_proj","k_proj","o_proj"]`) and converted with MediaPipe's `converter.ConversionConfig()`.

Model selection: validate LoRA on the target device *before* committing. Two candidates:

- **Gemma-2 2B** — explicitly documented for LoRA, larger footprint.
- **Gemma-3 1B (4-bit)** — smaller and faster, but LoRA support is not clearly documented for it.

Spike this in Phase 2 and let the measurement decide. If LoRA proves unworkable, the fallback is few-shot prompting + a stricter repair pass, and the portfolio narrative shifts from "fine-tuned" to "evaluated fine-tuning and shipped the alternative, with numbers" — still a legitimate result, but decide it on evidence rather than at the first error.

Structured output. MediaPipe has no grammar-constrained decoding (unlike llama.cpp's GBNF), so schema adherence is probabilistic and must be defended in layers:

1. Tight few-shot prompt with the schema inline.
2. Low temperature.

3. Extract the first balanced `{...}` span from the output.
4. Repair pass: trailing commas, unquoted keys, markdown fences.
5. Validate against the schema; on failure or `confidence < τ`, drop to T3.

Making step 5 fire rarely is the primary justification for fine-tuning at all.

4.5 Model lifecycle

Model weights are too large to bundle in an APK — they must be downloaded at runtime.

- First run: prompt the user, download over unmetered network via `WorkManager`, verify SHA-256, store in app-private storage.
- The LoRA adapter is small (single-digit MB) and may be bundled or shipped alongside.
- The launcher is **fully functional at T0/T1/T3 before the model exists**. SLM features degrade gracefully and announce themselves when ready.
- Hosting: **Azure Blob Storage** (free tier), which replaces the static hosting that Static Web Apps incidentally provided in v1. Gemma weights are license-gated on Hugging Face, so either require the user to accept the license and side-load, or host the converted artifact yourself — confirm the license permits redistribution first, and check that free-tier egress survives a multi-GB download. *(Open items — see §11.)*

4.6 Note / checklist editor

Changed in v3. The v2 design specified a vim-style modal editor (NORMAL/INSERT/COMMAND, `hjkl`, `:w`/`:q`). That is replaced by a **Google Keep-style touch editor rendered inline in the CLI surface**. The command-line entry point is unchanged — `note standup` still opens the editor from the prompt — but what appears is a direct-manipulation editor, not a modal one.

Rationale: a vim emulator is a large amount of work whose payoff is ergonomics for a single user who must already be holding a phone. Touch is the native input method of the device; fighting that to preserve CLI purity costs weeks and produces something slower to actually write a note in. **This change removes more work than it adds.**

Inline, not a separate Activity. The editor is a Compose overlay composited over the launcher surface within the same Activity — the transcript remains visible behind/above it. No `startActivity`, no window transition, no separate task in Recents. Dismissing it returns to the prompt with focus restored to the input line.

Layout

```
+-----------------------------------------+
|  Title                       [x]  [check]  |  <- action bar
+-----------------------------------------+
|  Body text, freely editable               |
|                                           |
|  [ ] checklist item                       |
|  [x] completed item                       |
|  +  Add list item                         |
+-----------------------------------------+
|  #work  #urgent            +   ...        |  <- tags, add, overflow
+-----------------------------------------+
```

- **Top-right actions:** x closes, check confirms and closes. Overflow carries delete, convert note ↔ checklist, and hide-checked-items.
- **Checklist items** are individually checkable, reorderable by drag, and deletable. Checking an item moves it below the unchecked ones (Keep's behaviour) rather than leaving it in place.
- **Tags** are chips at the bottom, and inline `#work` in the body is still parsed — both routes produce the same `tags` array.

Autosave is the real save mechanism. The buttons stay, because an editor with no visible commit affordance feels unfinished — but they are *not* what protects the data. Every change is debounced and written to Room continuously, so a killed process, a phone call, or a swipe-away loses nothing. The check button means "done", not "commit". This costs almost nothing given the outbox already exists (§7.1), and it is how Keep itself behaves.

Keyboard path preserved. The premise of the project is keyboard-first, so the editor must not force a reach for the screen: `Esc` closes, `Ctrl+Enter` confirms and closes, `Tab`/`Enter` moves between checklist items. A note started from the keyboard should be finishable from the keyboard.

Storage format is still Markdown — but note the rationale has shifted. In v2 the argument was that a rich-text JSON model (Quill Delta, ProseMirror) is hostile to a plain-text modal editor. With the modal editor gone, that argument is weaker. Markdown stays anyway, for reasons that still hold: desktop parity with the WPF Markdig editor (§5.3), clean diffing, portability, and no proprietary lock-in. Checklists remain `- [ ]` / `- [x]` lines, parsed into structured items for both UIs — so the Keep-style checkbox list and the desktop checkbox list are two views of one Markdown body.

Tags are inline hashtags (`#work`) parsed out of the body on save and denormalized to a `tags` array for query.

4.7 Aliases

`alias gm = create_note "standup notes" #work`. Stored locally in Room, resolved at T0, listed via `:alias`. Aliases are device-local and not synced (v1) — they encode device-specific app choices.

---

5. Desktop client (WPF)

Native Windows application, **.NET 10 + WPF**, MVVM throughout. This is a deliberate portfolio choice: WPF is Windows-only and unfashionable, but it appears in far more enterprise .NET job postings than WinUI 3 or MAUI, it ships with .NET 10, and it is in better shape in 2026 than it has been for years (Fluent theme, Store distribution). Skills transfer cleanly to Avalonia later if cross-platform ever matters.

5.1 Stack

Concern	Choice	Why
UI	WPF (.NET 10)	Target job market; mature XAML/data-binding story
Pattern	MVVM via `CommunityToolkit.Mvvm`	Source-generated observables; no hand-rolled `INotifyPropertyChanged` boilerplate
DI / host	.NET Generic Host	Same DI and configuration idioms as ASP.NET Core — the transferable part
Local store	**EF Core + SQLite**	Offline source of truth; migrations; the most quoted .NET data skill
HTTP	`HttpClient` via `IHttpClientFactory` + Polly	Retry/backoff on a flaky link is the whole point of an offline client
Auth	**MSAL.NET** public client, PKCE	See §6.4

Concern	Choice	Why
Markdown	Markdig	Rendering and round-trip parsing
Tests	xUnit + FluentAssertions	Sync engine is the thing worth testing

5.2 Structure

Three projects, and the middle one is the interesting one:

```
Notes.Shared     — POCOs, DTOs, tag parsing, merge rules, sync contract
Notes.Desktop    — WPF app: views, viewmodels, EF Core, sync service
Notes.Functions  — C# isolated-worker Azure Functions
```

`Notes.Shared` is referenced by both the desktop app and the Functions project, so the document model and merge rules are defined **once** and cannot drift between client and server. Android reimplements the same contract in Kotlin — that boundary is documented in `Notes.Shared` as the canonical spec, and contract tests on both sides assert agreement.

5.3 UI

- Two-pane: left is a scrollable note list sorted by `updatedAt` desc, filterable by category; right is the editor.
- Markdown editor with live preview. Checklist items are clickable checkboxes that rewrite the underlying `- [ ]` line.
- Category filter is a multi-select over the distinct `tags` set, computed locally from SQLite — instant, no round trip.
- Light **and** dark themes (unlike Android) — this is a desktop reading surface, and the asymmetry is intentional. WPF's Fluent theme supports both; bind to the system preference with a manual override.
- A visible sync status affordance: last-synced timestamp, pending-change count, and any conflict notice. An offline-first app that hides its sync state feels broken rather than offline.

5.4 Offline behaviour

SQLite is the source of truth. **The UI never awaits the network.** All reads and writes hit EF Core; a `SyncService` running as an `IHostedService` reconciles in the background on a timer, on reconnect (`NetworkChange.NetworkAvailabilityChanged`), and on demand.

Every local mutation is written to an outbox table in the same transaction as the entity change. Sync drains the outbox. This makes an interrupted sync safe to resume and is a genuinely defensible pattern to talk about in an interview.

5.5 Distribution

No hosted URL any more — this needs a real install story. `dotnet publish` self-contained, single-file, then **MSIX** packaged for install and update. Overkill for one user, deliberately: packaging and signing is exactly the tedious part most side projects skip and most employers care about.

---

6. Azure backend

6.1 Services and free-tier envelope

Service	Plan	Free allowance	Constraint that matters
Azure Functions	Consumption	1M executions + 400,000 GB-s / month, ongoing	Applies to paid (pay-as-you-go) subscriptions
Cosmos DB (NoSQL)	**Provisioned**, free tier	1000 RU/s + 25 GB, lifetime	**Not available on serverless accounts**; one per subscription; opt-in **at creation only**
Blob Storage	Free/LRS	Small monthly allowance	Hosts the SLM + LoRA artifacts for Android's first-run download
Azure AI Language	F0	Free monthly record allowance	Optional; verify current allowance before relying on it

Static Web Apps is **removed** in v2 — there is no web client left to host.

> **Trap worth stating explicitly:** "serverless" applies to *compute*, not the database. Use Functions Consumption (serverless compute) **with Cosmos DB in provisioned throughput mode**. Selecting Cosmos DB's serverless capacity mode silently forfeits the free tier, and it cannot be enabled after account creation — the account must be recreated.

Guardrails: a $0 budget alert on the subscription, and Cosmos configured with **manual** provisioned throughput pinned at 1000 RU/s — not autoscale, whose max-throughput ceiling bills above the free grant the moment it scales.

6.1a Runtime and language

C# on the Functions isolated worker model, .NET 10. Not negotiable, and not merely a preference:

> ⚠ **Hard deadline.** Support for the Functions **in-process** .NET model ends **10 November 2026** — under three months from this document's date. Any tutorial, sample, or Stack Overflow answer predating this is likely to show the in-process model. Start on the isolated worker model; do not migrate to it later.

Isolated worker is also the better story anyway: a real `Program.cs`, standard DI via `Microsoft.Extensions.*`, middleware, and full control of the host — the same idioms as the WPF app's generic host and any ASP.NET Core job you'd take.

6.2 Document model

```
{
  "id": "uuid-v4",
  "userId": "single-user-id",
  "type": "note | checklist",
  "title": "Standup notes",
  "body": "markdown source, hashtags inline",
  "tags": ["work"],
  "createdAt": "2026-08-13T09:00:00Z",
  "updatedAt": "2026-08-13T09:42:11Z",
  "deleted": false,
  "sourceDevice": "pixel-8",
```

```
  "rev": 7
}
```

`rev` is new in v2: a monotonically increasing revision assigned by the server on each accepted write. With two offline-capable clients, `updatedAt` alone is not enough to reason about causality when device clocks disagree — `rev` gives sync a clock-independent ordering (§7).

Partition key `/userId`. For a single user this is one logical partition (20 GB cap) — degenerate by design, and correct at this scale. Note the tradeoff in the README; a real multi-tenant design would partition differently, and knowing *why* is the interesting part.

Checklist items are derived from the Markdown body, not stored separately — one source of truth.

6.3 API

HTTP-triggered Functions, Cosmos bindings where they fit:

```
GET    /api/notes?since={rev}&tag={t}  → changed docs since watermark
GET    /api/notes/{id}
PUT    /api/notes/{id}                 → upsert, If-Match: {etag}
DELETE /api/notes/{id}                 → soft delete (tombstone)
POST   /api/sync/batch                 → batch push + pull        [v2]
POST   /api/categorize                 → { body } → { tags[] }    [opt]
```

`POST /api/sync/batch` is added in v2. A desktop client returning from a long offline stretch may have dozens of pending changes; one round trip per note wastes both executions and RU/s against a fixed free grant. Batch push-and-pull in a single call, with a `rev` watermark, keeps sync to roughly one execution per cycle regardless of backlog size.

DTOs and the sync contract are defined in `Notes.Shared` and referenced directly by `Notes.Functions`, so the desktop client and server cannot disagree about the wire format.

6.4 Auth

Single user, but the API is on the public internet and must not be open. The v1 answer leaned on Static Web Apps' built-in auth, which no longer exists here — so both clients now do real OAuth.

- **Desktop: MSAL.NET** (`Microsoft.Identity.Client`) as a *public client* using the authorization code flow with PKCE — no client secret, because a desktop binary cannot keep one. Tokens cached via MSAL's encrypted Windows token cache; silent refresh on launch.
- **Android:** MSAL Android (or AppAuth) against the same Entra ID app registration, refresh token in `EncryptedSharedPreferences` backed by the Android Keystore.
- Functions validate the JWT (issuer, audience, signature, expiry) in middleware and reject anything without a matching `userId`.

Registering one Entra ID application with two platform configurations — a public desktop client and an Android client — is a small amount of work and a disproportionately good thing to be able to discuss in a .NET interview. A pre-shared API key remains the low-effort fallback, and is *adequate* for single-user personal use, but it rotates badly and undersells the project; if you take that path, say so plainly in the README rather than implying otherwise.

Functions validate the JWT and reject anything without a matching `userId`.

6.5 Auto-categorization (optional)

Correcting the original scope: sentiment analysis returns positive/negative/neutral. It does not produce categories like `#work`. What the scope actually describes is **text classification**.

Options, cheapest first:

1. **Keyword/heuristic rules** — free, deterministic, probably sufficient for a personal tag vocabulary.
2. **Azure AI Language custom text classification (F0)** — real ML, needs a labeled training set.
3. **Reuse the on-device SLM** — classify at note-creation time on the phone, zero cloud cost, and reuses infrastructure you've already built.

Option 3 is the most interesting and the most coherent with the rest of the project. Treat categorization as a stretch goal either way; it is not on the critical path.

---

7. Sync

This section changed most in v2, and it is now the second-riskiest part of the project after the SLM.

In v1 the desktop was a thin client, so only one replica (the phone) could diverge and conflicts were largely theoretical. With a native offline-capable desktop there are **two independent replicas that can both accept writes while partitioned**. Concurrent conflicting edits stop being a hypothetical and become something you will actually hit — editing a note on your phone on the train, then on your desktop before the phone syncs, is a Tuesday.

7.1 Shape

Symmetric, local-first on both clients. Local store is the source of truth; the network is an optimization.

- Both clients write mutations to an **outbox** in the same transaction as the entity change (Room on Android, EF Core/SQLite on desktop). Sync drains the outbox; an interrupted sync is safe to resume.
- Sync cycle: push outbox → pull `?since={rev}` → apply → advance watermark. One `POST /api/sync/batch` round trip per cycle.
- Triggered on a timer, on connectivity regain, on app foreground, and on demand.
- Soft deletes only; tombstones propagate deletion. Purge tombstones older than 90 days.
- The server assigns `rev`. Clients never invent one. This gives a total order that does not depend on either device's clock being correct.

7.2 Conflict policy

Last-write-wins on server `rev`, with the loser preserved as a conflict copy.

This is the v2 change worth defending. Plain LWW silently destroys the losing edit, which was tolerable when only one replica could diverge and is not tolerable now. Instead: when a client pushes a change whose `baseRev` is older than the server's current `rev` for that document, the server accepts the newer write and returns a conflict marker; the client then materialises the losing version as a **separate note** titled `<title> (conflict, <device>, <timestamp>)`.

Cheap to implement, impossible to lose data with, and honest about the tradeoff — the user resolves the merge by hand, which for a personal notes tool is the right division of labour. It's also the same thing Dropbox does, which makes it easy to explain.

`If-Match` on the Cosmos ETag still guards the interleaved-write race at the server and triggers a re-pull.

7.3 Shared contract

Merge rules, `rev` comparison, tag parsing, and the DTOs live in `Notes.Shared` and are consumed directly by both `Notes.Desktop` and `Notes.Functions`. Android reimplements them in Kotlin.

That reimplementation is the seam where bugs will live. Mitigate with a **shared fixture set** — a JSON corpus of sync scenarios (`base`, `local`, `remote`, `expected`) committed once and executed by both the xUnit suite and the Android unit tests. If the two implementations disagree, a test fails rather than your notes quietly diverging.

7.4 Explicitly not doing

CRDTs are the correct general answer to this problem and are **out of scope**. For a single user with occasional conflicts, a conflict-copy strategy delivers most of the value for a fraction of the effort. Being able to say exactly that — and why you'd choose differently for a multi-user product — is worth more in an interview than a half-finished CRDT.

---

8. Evaluation plan

Without measurements this is a demo, not an engineering project. These artifacts are deliverables:

Dataset. 800–1,500 command examples: synthetic generation across intent × phrasing × slot variation, plus real T0/T1 logs from §4.2, hand-labeled. Held-out test split frozen before any tuning. Include adversarial/out-of-scope inputs mapped to `unknown`.

Metrics.

Metric	Method
Intent accuracy, per-intent F1	Held-out test set
Slot exact-match	Held-out test set
Schema-valid JSON rate	Pre- and post-repair
Abstention quality	FPR on out-of-scope inputs
Time-to-first-token, tokens/sec	On-device, instrumented
Peak RSS during inference	`adb shell dumpsys meminfo`
Battery / thermal	Sustained 50-command loop

Baselines to beat (and report even where you lose): deterministic parser alone; base model with few-shot, no LoRA; fine-tuned model. The comparison table *is* the portfolio artifact — more so than the app.

Sync correctness is the second test surface, and unlike the ML metrics it is pass/fail rather than a number:

Scenario	Expectation
Edit offline on desktop only	Converges on reconnect, no conflict copy
Edit offline on both, same note	Newer `rev` wins; loser preserved as a conflict copy
Delete on one, edit on the other	Tombstone wins; edited version preserved as a conflict copy
Sync interrupted mid-push	Outbox replays; no duplicates, no lost writes
Clock skew (device set an hour ahead)	Ordering unaffected — `rev` is server-assigned
Shared fixture corpus	Kotlin and C# implementations produce identical results

Automate these in xUnit against an in-memory/SQLite harness. They are far cheaper to run than the ML benchmarks and will catch far more real bugs.

---

9. Phased delivery

Phase	Deliverable	Exit criterion
0	Launcher shell + T0/T1/T3, app index, aliases, settings intents	Daily-drivable as your real home screen, zero ML
1	Keep-style inline editor + Room, local notes only	Notes usable offline end-to-end on Android
2	**Spike:** MediaPipe + base model on device, few-shot prompting, `InferenceEngine` interface	End-to-end NL command works once; LoRA viability answered
3a	`Notes.Shared` + `Notes.Functions` (isolated worker) + Cosmos + Entra ID auth	Authenticated CRUD against the cloud from a REST client
3b	WPF shell + EF Core/SQLite + MVVM, local-only, no sync	Desktop usable offline as a standalone notes app
3c	`SyncService` + outbox + conflict copies, both clients	Sync scenario table in §8 passes
4	Dataset + LoRA fine-tune + quantize + benchmark suite	Benchmark table populated
5	MSIX packaging, README, architecture diagram, writeup	Installable and presentable
v2	LiteRT-LM engine + backend comparison	Both backends benchmarked

Phase 3 is split in v2 because it roughly tripled in size — it is now a backend, a desktop application, and a distributed sync engine rather than "a web page and some Functions." Each sub-phase has an independent exit criterion so partial progress is still demonstrable.

Phase 0 is deliberately first and ML-free: it makes the thing genuinely useful before any of the risky work starts, and it generates the training data Phase 4 needs. Phase 2 is a **spike, timeboxed** — its job is to answer "does LoRA work on this device with this model," not to be good.

Sequencing note: 3b (WPF + EF Core, local-only) is the single best phase to do early if the .NET job hunt is time-sensitive. It is low-risk, entirely offline, demonstrable on its own, and it is the part a .NET hiring manager will actually look at. Nothing in it depends on the SLM or the backend. If the schedule slips, slipping Phase 4 costs you less than slipping 3b.

Hard timebox Phase 4. "Improve accuracy" has no natural stopping point. Set a date and a target metric in advance; ship whatever the number is on that date and write it up honestly.

---

10. Risks

Risk	Impact	Mitigation
Total scope grew materially in v2	**High**	The desktop went from a web page to a full offline application with its own store and sync engine. This is the main cost of the change and is worth accepting only because the .NET work *is* the point. Phase 3 split accordingly
Two offline replicas → real conflicts	High	Server-assigned `rev`, outbox pattern, conflict copies instead of silent LWW, shared fixture corpus (§7)
Kotlin and C# sync logic drift	Medium	Shared JSON fixture corpus executed by both test suites; failures surface as test failures, not data loss
Functions in-process model retires 10 Nov 2026	Medium	Start on the isolated worker model; treat pre-2026 samples as suspect
MediaPipe in maintenance-only mode	Medium	`InferenceEngine` abstraction; LiteRT-LM migration planned as v2, not as a fire drill
LoRA unsupported on chosen model/device	High	Phase 2 spike answers this early; few-shot fallback preserves the project
SLM latency makes the launcher feel slow	High	Tiered pipeline — SLM never sits on the fast path
Model download size / license gating	Medium	First-run download from Blob Storage, graceful degradation, resolve licensing before hosting
Cosmos free tier lost via wrong capacity mode	Medium	Provisioned mode at creation; recreating the account is the only fix
WPF perceived as legacy by some reviewers	Low	Frame it accurately: .NET 10, MVVM, DI, EF Core, MSIX. The patterns are what transfer, and they are current
Phase 4 scope creep	High	Hard date + pre-committed target metric
Being your only home screen while broken	Medium	Keep a second launcher installed; never ship an untested build to your daily driver

---

11. Open items

1. Verify current Azure AI Language F0 monthly allowance before depending on it.
2. Confirm Gemma license terms permit hosting the converted artifact in Blob Storage, or require user-side download.
3. Confirm Blob Storage free-tier egress is sufficient for occasional multi-GB model downloads

— a single re-download could dominate the allowance. If not, fall back to user-side download from Hugging Face.
4. Choose target device and pin it; all benchmarks are device-specific and must name it.
5. Confirm whether Gemma-3 1B supports LoRA under MediaPipe, or whether Gemma-2 2B is required.
6. Decide whether aliases sync (§4.7 currently says no). With a real desktop client the answer is less obvious — but desktop has no apps to launch, so probably still no.
7. Confirm Entra ID free tier covers a single-user app registration with two platform configurations (expected yes; verify before building auth).

Sources

- Lifetime Free Tier — Azure Cosmos DB
- Azure Functions pricing
- Migrate .NET Functions to the isolated worker model — in-process support ends 10 Nov 2026
- Differences between in-process and isolated worker .NET Functions
- WPF modernization in 2026 (Avalonia blog — author discloses vendor bias)
- MediaPipe LLM Inference guide
- MediaPipe LLM Inference guide for Android
- Fine-tuning Gemma with LoRA for on-device inference
- Blazing fast on-device GenAI with LiteRT-LM